Part – I: MODERN PYTHON & APPLIED DATA FOUNDATIONS

Introduction to Production Python and AI-Assisted Coding

- Introduction to Python; Environment Setup with Poetry; Loops and Conditionals; Strings and Lists; Tuples, Sets, and Dictionaries; Writing Reusable Functions; Structured Logging and Debugging Frameworks; Connecting Python to Enterprise SQL Databases (Snowflake, BigQuery); Writing Advanced SQL Window Functions; Prompt Engineering for AI Coding Assistants (GitHub Copilot, Cursor); Basic Assignments
- Homework

Advanced Python, Code Architecture, and Applied Business Statistics

- Advanced Python: Object-Oriented Programming (OOP) for Machine Learning Pipelines; Class and Object; Modules and Custom Exception Handling; Abstractions and Inheritance; Unit Testing with `pytest`
- Applied Statistics: Population vs. Sample; Categorical and Numerical Data; Descriptive Statistics; Advanced Hypothesis Testing for Product Management; A/B Testing Design; Type I and Type II Errors; Correlation vs. Causation; Statistical Data Drift Metrics (Population Stability Index)
- Python Projects and Homework
- **Project 01:** Automated E-Commerce SQL Analytics Pipeline with Robust Exception Handling and AI-Assisted Optimizations

Part – II: APPLIED MACHINE LEARNING & MLOps

Core Predictive Modeling and Experiment Tracking

- Introduction to Modern Machine Learning: Supervised, Unsupervised, and Ensemble Techniques; Moving from Scratch Coding to Production Frameworks (`scikit-learn`, `XGBoost`, `LightGBM`)
- Gradient Descent Intuition; Linear Regression Implementation; Regularization Techniques (Ridge and Lasso)
- Binary and Multi-Class Classification: Logistic Regression; Evaluating Models with Confusion Matrix, Accuracy, Precision, Recall, and F1-score
- Tree-Based Architectures: Decision Trees; Ensemble Models; Bootstrap Aggregation (Bagging); Boosting Concepts; Random Forest and Extreme Gradient Boosting (XGBoost)
- Unsupervised Learning: K-Means Clustering; Dimensionality Reduction for High-Dimensional Corporate Data

Production Feature Engineering and Experiment Management

- Modern Feature Engineering: Advanced Imputation Pipelines; One-Hot Encoding vs. Target Encoding; Outlier Handling (Robust Z-Score, Quantile-based capping); Feature Scaling Strategies
- MLOps Lifecycle: Replacing Manual Tracking with Automated Experimentation Frameworks; Introduction to **MLflow** and Weights & Biases
- Tracking Hyperparameters, Code Versions, and Model Artifacts; Utilizing Model Registries for Version Control
- Theory and Live Coding Demonstration; Projects and Homework
- **Project 02:** Enterprise Customer Churn Prediction Pipeline with End-to-End MLflow Experiment Tracking and Model Registration

Part – III: DEEP LEARNING & FOUNDATION MODELS

Neural Network Infrastructure and the Hugging Face Ecosystem

- Foundations of Deep Learning: Neural Networks Architecture; Forward Propagation and Backpropagation Intuition; Activation Functions (ReLU, GELU, Sigmoid); Custom Loss Functions; Gradient Descent Optimizers (AdamW, RMSprop)
- Introduction to Modern PyTorch Framework: Tensors, Autograd, and Building Custom Data Loaders
- The Death of Custom LSTMs/RNNs: Transitioning to the Transformer Architecture; Multi-Head Attention Mechanisms; Self-Attention Intuition
- Hugging Face Ecosystem: Utilizing `transformers` and `datasets` Libraries; Downloading, Loading, and Inferencing Pre-trained Foundation Models for Text Classification, Named Entity Recognition (NER), and Sentiment Analysis
- Homework and Code Demonstrations

Production Computer Vision and Vision-Language Models (VLMs)

- Evolution of Vision Systems: Transfer Learning with Pre-trained Arcs (ResNet, EfficientNet); Introduction to the Modern **YOLO** Ecosystem for Object Detection
- Advanced Computer Vision: Fine-Tuning Open-Source Vision-Language Models (VLMs) for Automated Visual Parsing, Image Captioning, and Visual Question Answering (VQA)
- Text Pre-Processing for Modern NLP: Byte-Pair Encoding (BPE); SentencePiece Tokenizers; Replacing Word2Vec with Dense Contextual Embeddings
- Theory and Live Coding Demonstration; Projects and Homework
- **Project 03:** Automated Defect Detection in Manufacturing using Fine-Tuned Edge YOLO Models
- **Project 04:** Multi-modal Financial Document Parser using Hugging Face Vision-Language Transformers

Part – IV: ENTERPRISE GENERATIVE AI & ORCHESTRATION

Large Language Model Engineering and Advanced RAG

- Introduction to Enterprise Generative AI: Commercial APIs (OpenAI, Anthropic, AWS Bedrock) vs. Open-Source Models (Llama, Mistral); Tokenomics: Managing Token Budgets, API Latency, and Rate Limits
- Structured AI Output: Function Calling, JSON Mode, and Pydantic Schema Enforcement
- Advanced Retrieval-Augmented Generation (RAG): Building Production-Grade Knowledge Pipelines; Parsing Complex PDFs, Tables, and Charts; Document Chunking Strategies (Semantic Chunking, Parent-Child Relationships)
- Vector Infrastructure: High-Performance Embedding Generation; Indexing, Querying, and Hybrid Search (Sparse + Dense) in Vector Databases (**Pinecone**, **Milvus**, or **ChromaDB**)
- Search Optimization: Implementing Cross-Encoder Re-rankers to Improve Retrieval Context Relevance
- Orchestration Frameworks: Core Building Blocks of LangChain and LlamaIndex

Autonomous AI Agents and Multi-Agent Systems

- State Management in AI: Moving Beyond Linear RAG to Agentic Workflows; Introduction to **LangGraph** for Cyclic, State-Driven AI Systems
- Agent Tool Integration: Enabling LLMs to Dynamically Call SQL Databases, Browse the Web, and Execute Python Code Safely
- Multi-Agent Systems: Designing Collaborative Networks of Specialized Agents (e.g., Researcher Agent paired with a Writer Agent)
- Theory and Live Coding Demonstration; Projects and Homework
- **Project 05:** Enterprise Autonomous Market Research Agentic System Built with LangGraph, Tavily Web Search, and Structured JSON Data Output

Part – V: LLMOps, PRODUCTIZATION & CASE STUDIES

Microservice Development, Containerization, and AI Governance

- Industrial API Design: Building High-Performance, Asynchronous Backend Endpoints using **FastAPI** instead of Flask; Input and Output Data Validation with Pydantic
- System Containerization: Writing Production-Grade `Dockerfiles`; Local Microservice Multi-Container Orchestration using Docker Compose; Managing Python Dependencies using Poetry

- Automated GenAI Evaluation: Moving Away from Subjective Grading; Implementing Automated Frameworks (**Ragas**, TruLens) to Programmatically Score Faithfulness, Answer Relevance, and Context Recall
- AI Safety and Security: Implementing LLM Input/Output Firewalls (NeMo Guardrails, Llama Guard) to Prevent Prompt Injections, Secure Sensitive Data, and Block Hallucinations
- Continuous Integration: Basic GitHub Actions for Automatic Code Linting, Format Checking, and Basic Integration Testing

Industry Applications and Enterprise Deployment

- Industry Case Studies: Applied AI Systems in Retail (Intelligent Inventory Agents); Applied AI in Banking (Automated Compliance and Anti-Money Laundering Document Auditors)
- 20+ Industry-Relevant Code Frameworks covering Production MLOps, Hugging Face Implementations, Advanced RAG, and Agentic Systems
- 5+ End-to-End Scalable Multi-Container Project Deployments
- **Project 06 (The Final Graduation Blueprint):** Production-Grade Enterprise Knowledge Assistant — A Fully Dockerized FastAPI Microservice Utilizing Advanced Hybrid-Search RAG, LangGraph State Management, Semantic Caching for Cost Reduction, NeMo Safety Guardrails, and Ragas Automated Evaluation Frameworks